INGENIERÍA BIOMÉDICA

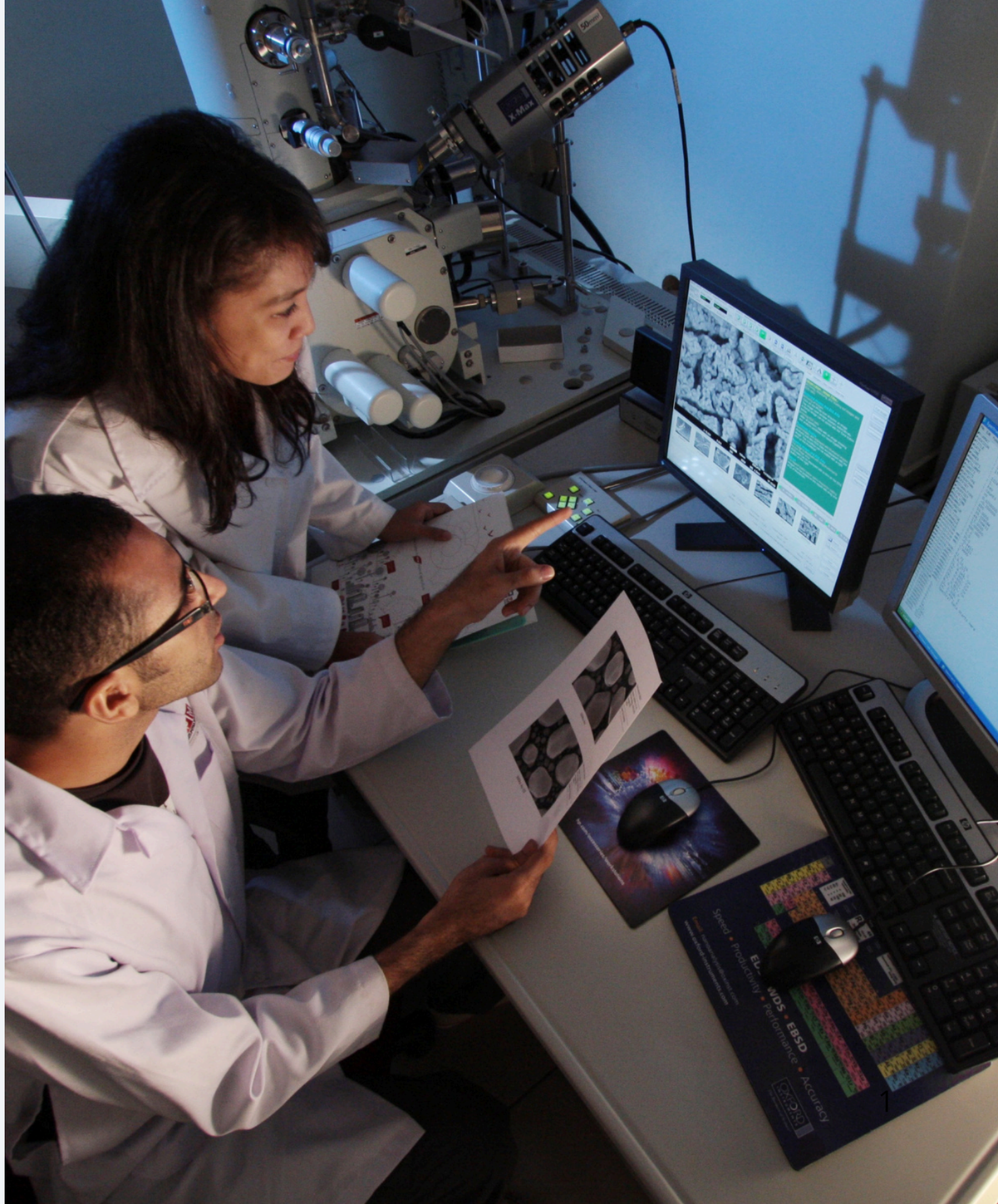
E10: METODOLOGÍA VDI: IMPLEMENTACIÓN

ENTREGABLE 10

GRUPO 3
FUNDAMENTOS DE BIODISEÑO
2026-2

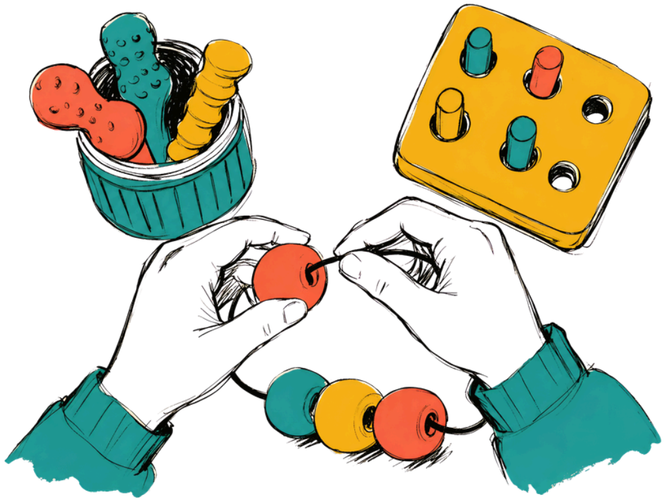
Munguia Avila, Ariana Alessandra
Martel Vilca, Dominick Pavel
Monzón La Rosa, María Isabel
Espinoza Cuya, Luis Diego
Guardamino Velasquez, Giovanni Jesus
Huanca Benites, Rodrigo Alonso

03/11/2026



¿Qué harías si no tuvieras movilidad de tu brazo dominante?





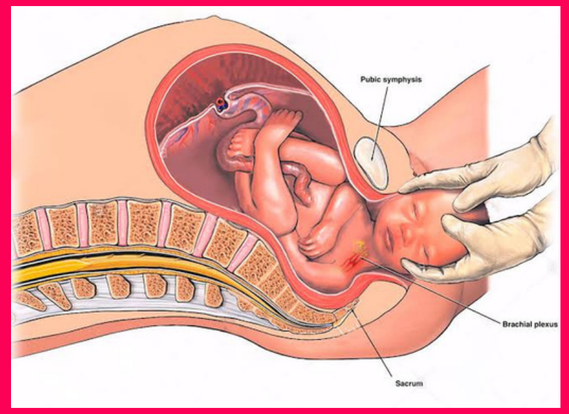
¿Qué harías si no tuvieras movilidad de tu brazo dominante?







UCC. 24 años



Según el INEI, el 43,7% al 45,2% de la población peruana con discapacidad forma parte de la fuerza laboral

Mayoría de personas con alguna discapacidad víctimas de hechos delictivos residen en Lima Metropolitana

De acuerdo con la información de la Encuesta Nacional de Programas Presupuestales (ENAPRES), el **15,6% de las personas con discapacidad**

INSEGURIDAD = menor calidad de vida

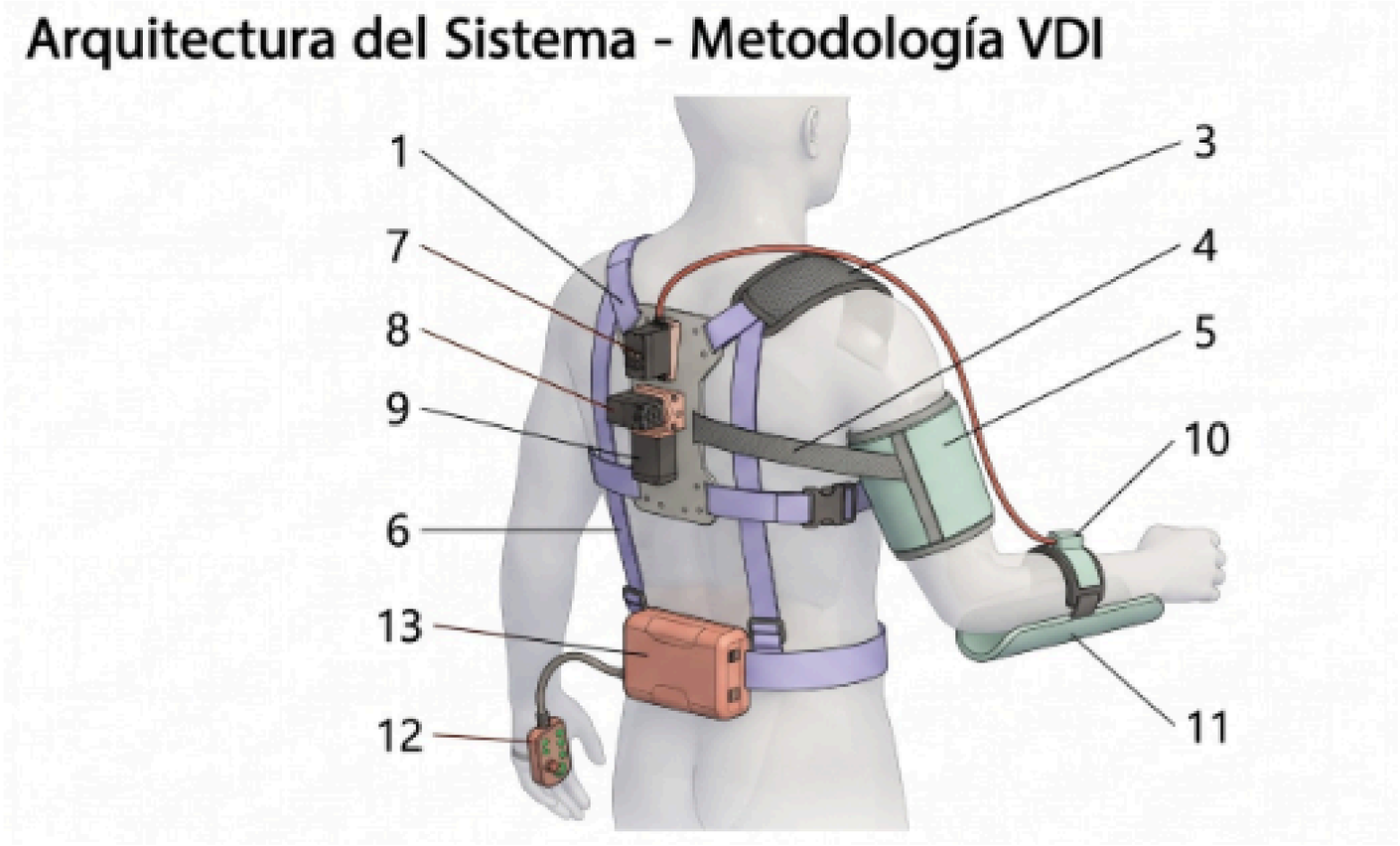
86 de cada 100 personas con algún tipo de discapacidad tienen seguro de salud

La cobertura del seguro de salud muestra que el 86,4% de la población con alguna discapacidad cuenta con algún seguro de salud. Según tipo de seguro, el **62% tiene Seguro Integral de Salud (SIS) y 23,1% con Seguro Social de Salud (EsSalud).**



https://www.gob.pe/institucion/inei/noticias/676448-el-52-7-de-la-poblacion-que-tiene-alguna-discapacidad-es-adulta-mayor

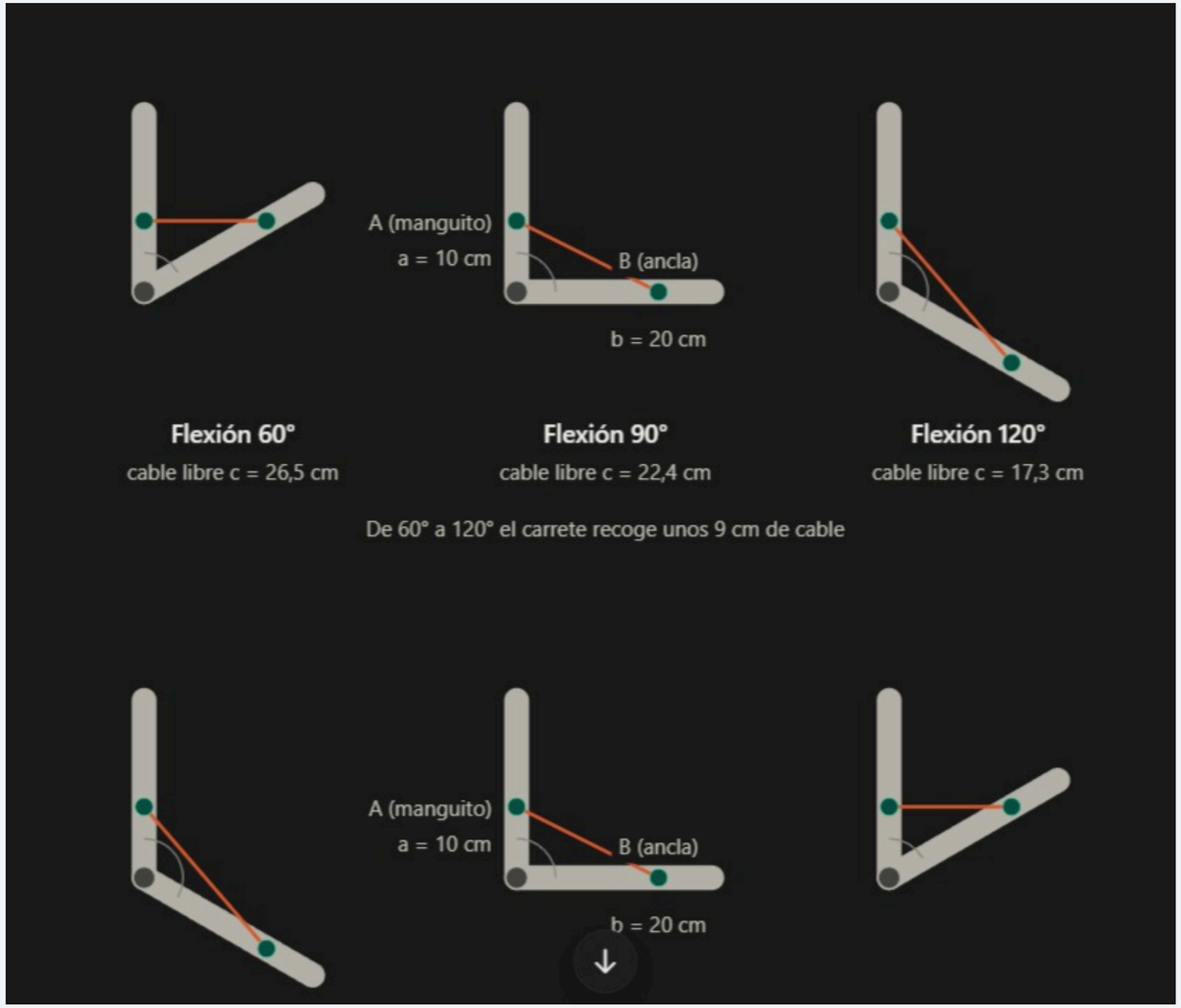
RESTRUCTURACIÓN DEL BOCETO



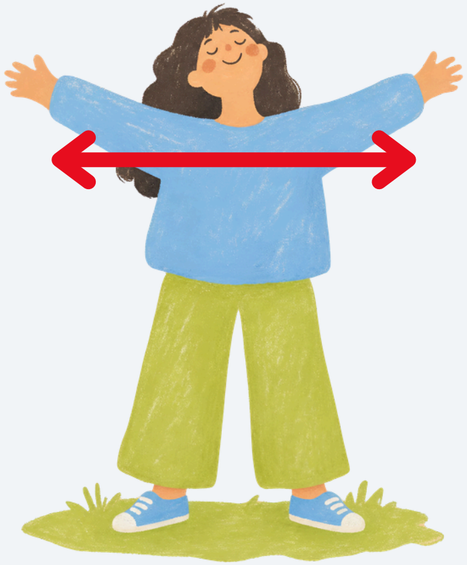
Partes del sistema

3. Correa de hombro acolchada
5. Banda del brazo
7. Servo del codo
8. Servo de rotación externa
10. “Ancla del modelaje”: permite conexión
11. Soporte del antebrazo
12. Control remoto
13. Baterías y sistema eléctrico
1-4-6. Sistema de correas.

RESTRUCTURACIÓN DEL BOCETO:
MOVIMIENTOS DE "MODO FUNDAMENTAL" Y "ROTACIÓN DEL CODO"

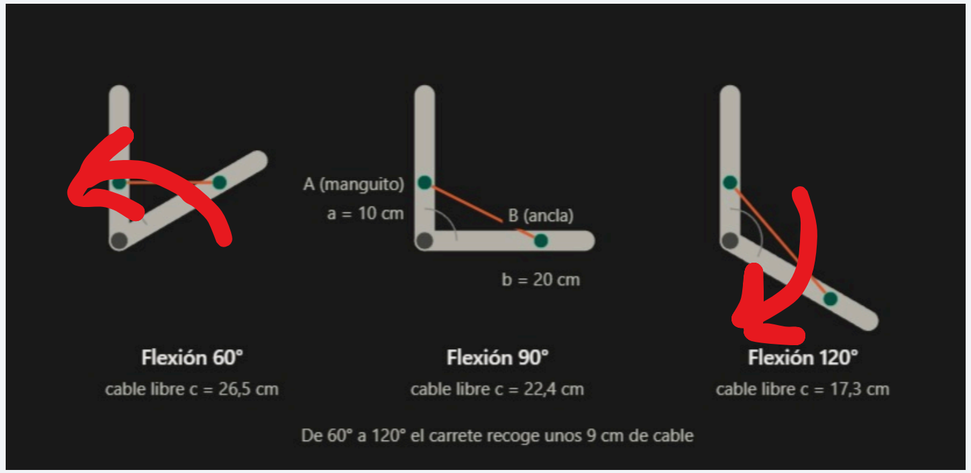


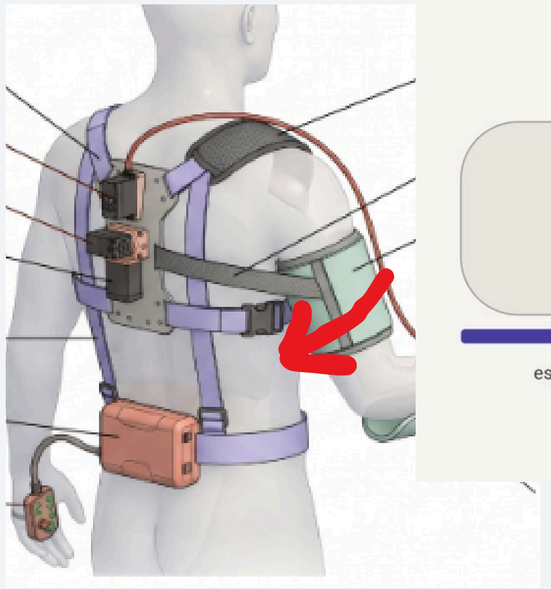
Evitar flexión del codo, considerando que la paciente normalmente puede elevar con ayuda a 30°

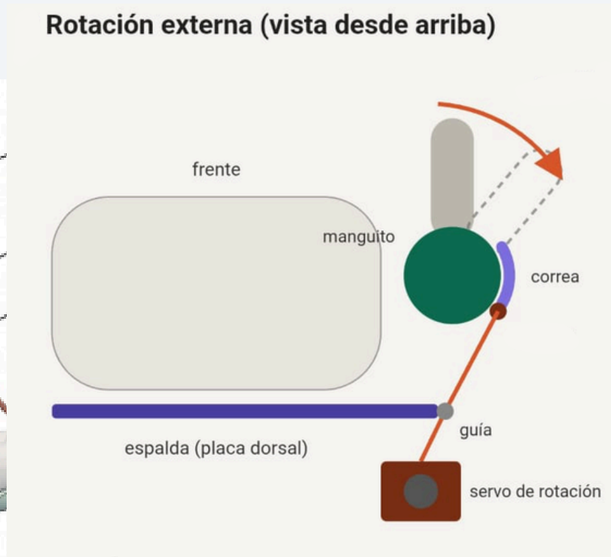


Avance progresivo

SISTEMA DE CORREAS

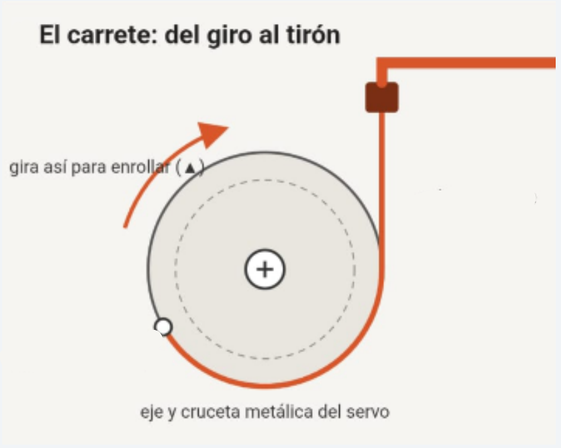






Movimientos en relación a **rotación del codo**:

1. El **servo gira y el carrete convierte ese giro en la fuerza**
2. El servo enrolla el cable **dentro de una funda, permitiendo la elevación del antebrazo.**
3. Para bajar el codo, el **servo desenrolla y la gravedad baja el antebrazo**



Movimientos en relación a **rotación externa del hombro**:

1. Otro servo **tira de la correa lateral fijada a la banda del brazo**, por lo cual el húmero gira y el antebrazo **rota de 30 a 45°.**

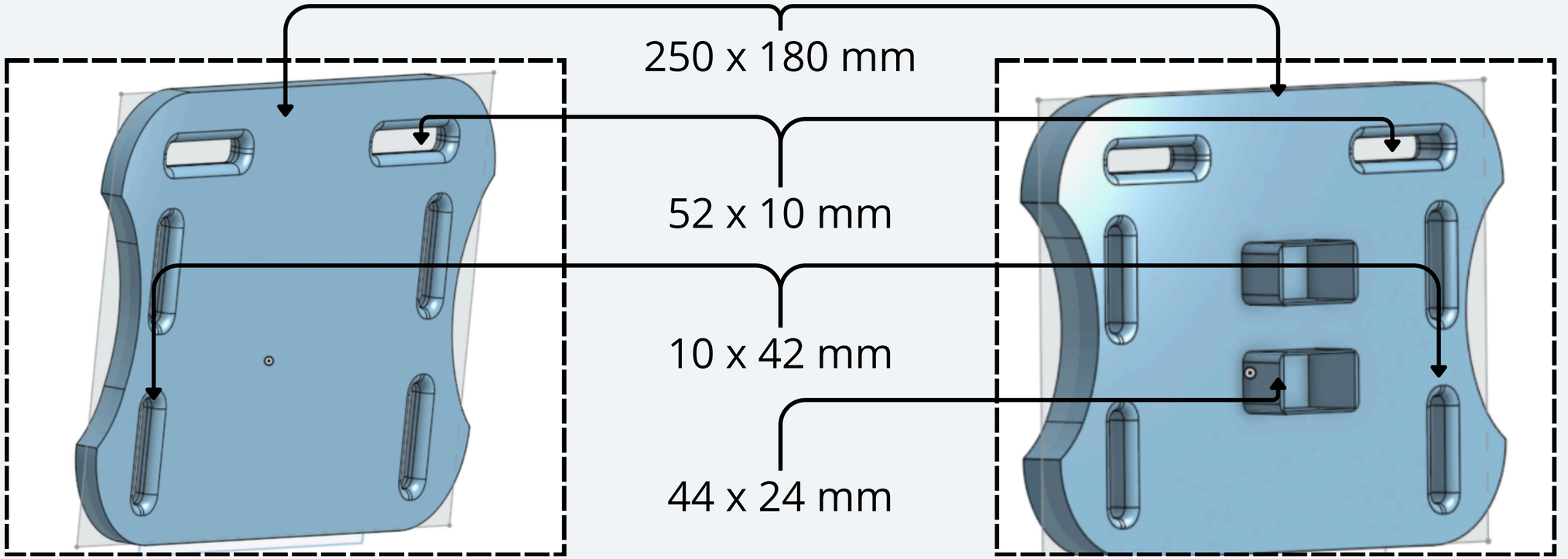
Una observación al respecto es que los cables solo tiran, no empujan el movimiento o la órtesis. Por lo mismo, al **“soltar”, la tensión desaparece y el brazo regresa solo.**

IMPLEMENTACIÓN PRELIMINAR DE LOS SUBSISTEMAS SEGÚN EL DISEÑO

MECÁNICA Y FABRICACIÓN

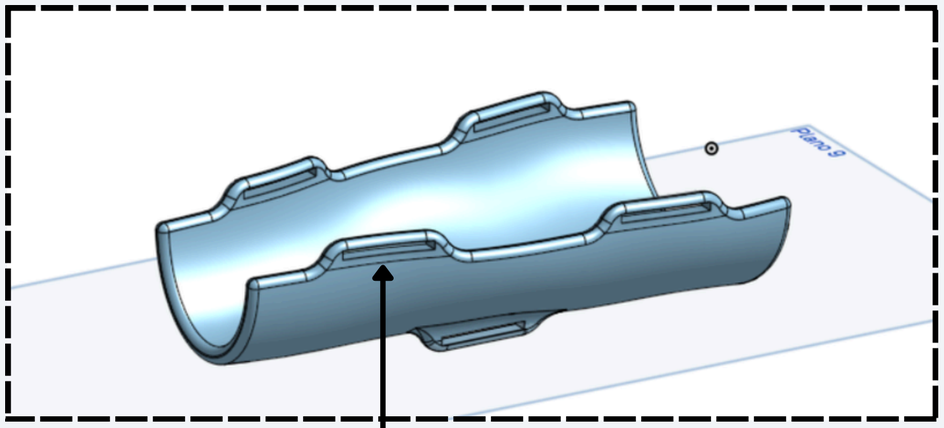
Parte frontal de la pechera

Parte Dorsal de la pechera



MECÁNICA Y FABRICACIÓN

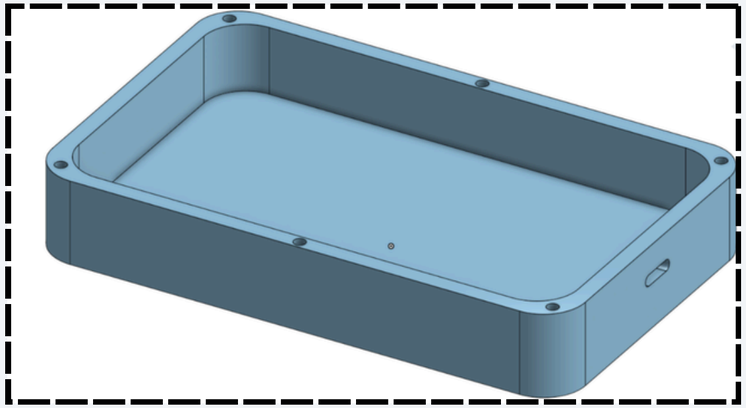
Soporte de Antebrazo



27.5x2.75 mm

LARGO:19.5 MM
ANCHO: 92.6 MM
ALTO: 33.5 MM

Caja de circuitos

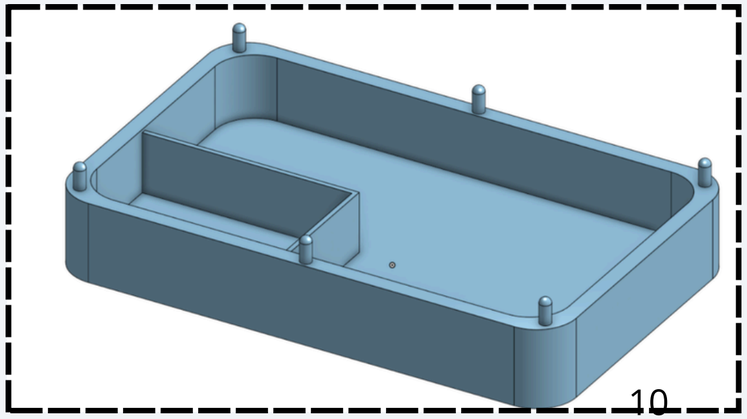


Ranura de carga tipo C

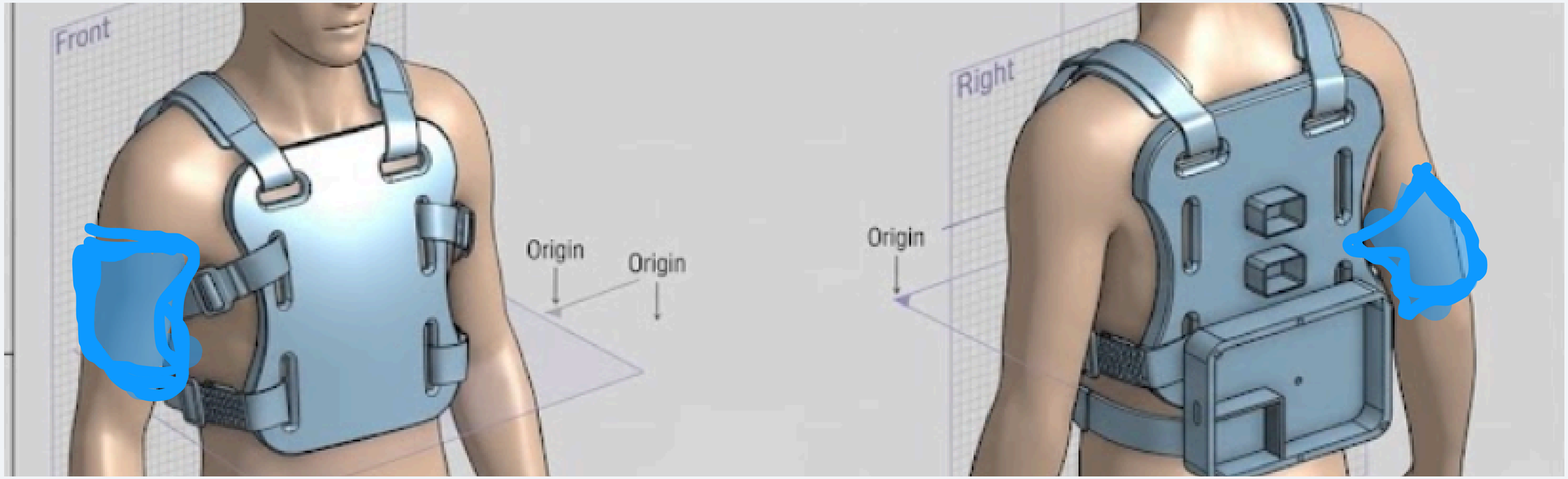
LARGO:12.4 MM
ANCHO: 6.8MM
PROFUNDIDAD: 5MM

LARGO:160 MM
ANCHO: 100MM
ALTO: 2.5MM

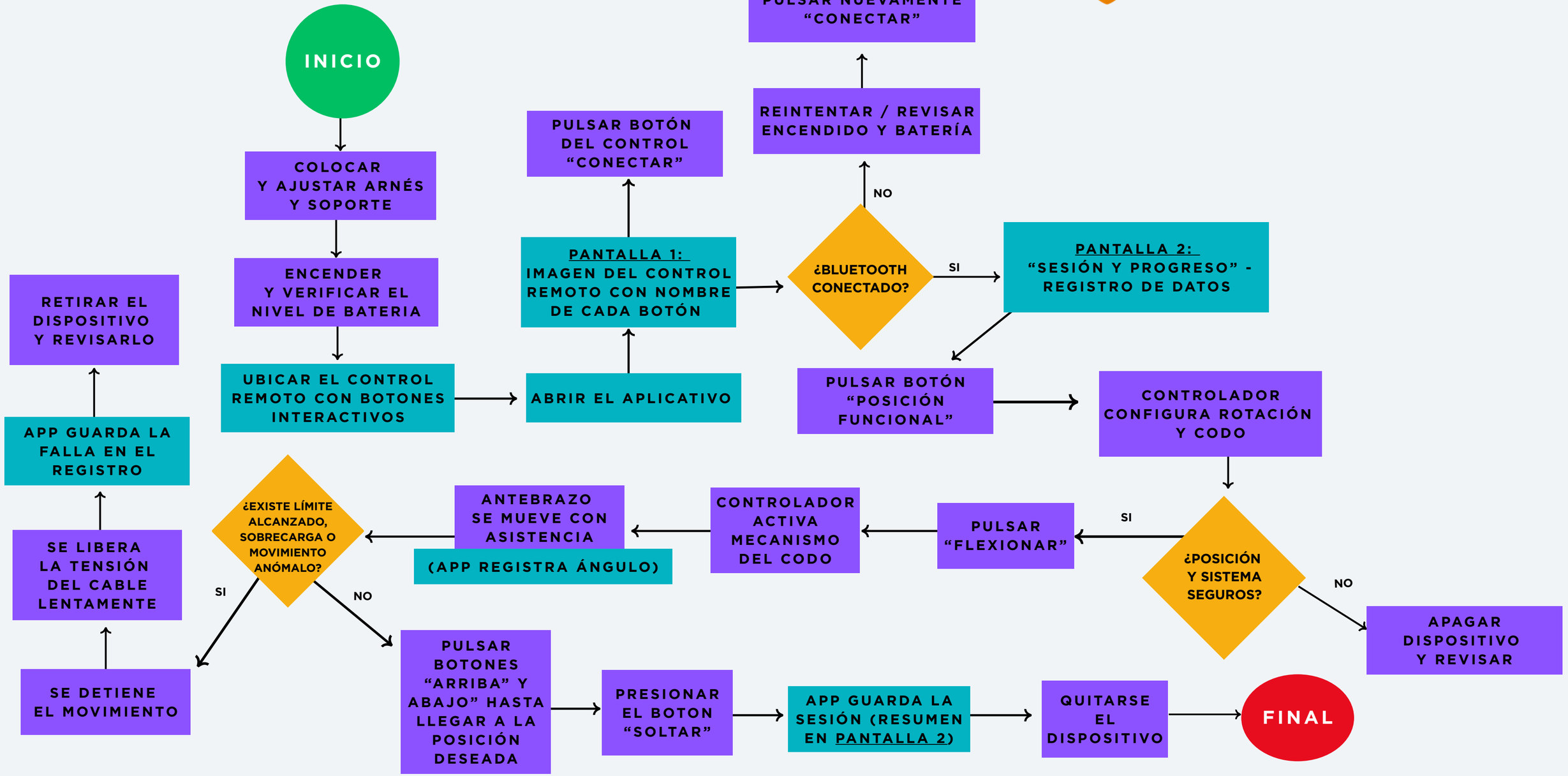
Ranura de bateria
Largo: 66.5 mm
Ancho: 37.5mm
Altura: 19mm



ENSAMBLADO DEL SISTEMA SUPERIOR (TORSO)



PROGRAMACIÓN Y CONTROL



ELECTRÓNICA Y SENSORES

ORTESIS_SERVOS_copy_20261104153844.ino

```
1
2   #include <Arduino.h>
3   #include <Wire.h>
4   #include <math.h>
5   #include <string.h>
6   #include <stdlib.h>
7   #include <ESP32Servo.h>
8   #include <Adafruit_INA219.h>
9   #include <Preferences.h>
10
11  const uint8_t PIN_SERVO_CODO = 19;
12  const uint8_t PIN_SERVO_ROT  = 18;
13  const uint8_t PIN_BTN_POS    = 25;
14  const uint8_t PIN_BTN_UP     = 26;
15  const uint8_t PIN_BTN_DOWN   = 27;
16  const uint8_t PIN_BTN_REL    = 14;
17  const uint8_t PIN_LED        = 13;
18  const uint8_t PIN_BUZZ       = 23;
19  const uint8_t PIN_SDA        = 21;
20  const uint8_t PIN_SCL        = 22;
21
22
23  const float A_CM       = 10.0f;
24  const float B_CM       = 20.0f;
25  const float SPOOL_R_CM = 2.2f;
26
27  const float ELBOW_SERVO_RANGE_DEG = 270.0f;
28  const float ROT_SERVO_RANGE_DEG   = 180.0f;
29  const int   SERVO_US_MIN = 500;
30  const int   SERVO_US_MAX = 2500;
31
32  const float THETA_REST = 50.0f;
33  const float THETA_MIN  = 60.0f;
34  const float THETA_MAX  = 120.0f;
```

ORTESIS_SERVOS_copy_20261104153844.ino

```
35
36  const float ROT_REST_DEG = 0.0f;
37  const float ROT_MAX_DEG  = 90.0f;
38
39  const float ELBOW_SPEED     = 15.0f;
40  const float ELBOW_REL_SPEED = 25.0f;
41  const float ROT_SPEED       = 15.0f;
42  const float ROT_REL_SPEED   = 25.0f;
43
44
45  const float    I_LIMIT_DEFAULT_MA = 2500.0f;
46  const uint16_t OVERCURRENT_MS     = 400;
47  const float    RAIL_MIN_V         = 4.5f;
48  const uint16_t RAIL_LOST_MS       = 300;
49  const bool     ALLOW_NO_INA219    = false;
50
51
52  const uint16_t LOOP_MS     = 20;
53  const uint16_t DEBOUNCE_MS = 30;
54  const uint16_t LONG_MS     = 3000;
55
56
57  enum State : uint8_t { ST_REST, ST_GO_ROT, ST_GO_ELBOW, ST_ASSIST, ST_RELEASING, ST_FAULT };
58  const char* STATE_NAME[] = {"REPOSO", "ROTANDO", "FLEXIONANDO", "ASISTENCIA", "LIBERANDO", "FALLA"};
59
60  Servo servoE, servoR;
61  Adafruit_INA219 ina;
62  Preferences prefs;
63
64  State state = ST_REST;
65  bool servosAttached = false;
66  bool inaOk = false;
67  bool lockout = false;
68  float elbowCur = THETA_REST, elbowTarget = THETA_REST;
69  float rotCur = ROT_REST_DEG, rotTarget = ROT_REST_DEG;
```

LIBRERIAS

- Wire.h: comunicación I2C con el INA219.
- math.h: sqrtf, cosf y fabsf, para la geometría del cable y los valores absolutos.
- string.h y stdlib.h: strcmp, strncmp y atof, para interpretar los comandos del monitor serie.
- ESP32Servo.h, Adafruit_INA219.h y Preferences.h: los servos, el sensor de corriente y la memoria permanente.

ELECTRÓNICA Y SENSORES

```
70  float thetaFunc = 90.0f;
71  float rotFunc = 40.0f;
72  float iLimitMa = I_LIMIT_DEFAULT_MA;
73
74  float currentMa = 0, railV = 0;
75  bool railOk = false;
76  float overMs = 0, railLostMs = 0;
77  bool railLost = false;
78  bool atLimit = false, limitBeeped = false;
79  bool posLongDone = false;
80  uint32_t lastTick = 0, lastStatus = 0;
81
82
83  struct Btn {
84    uint8_t pin;
85    bool state, lastRaw, pressedEdge, releasedEdge;
86    uint32_t tChange, tPress;
87    explicit Btn(uint8_t p)
88        : pin(p), state(false), lastRaw(false), pressedEdge(false), releasedEdge(false), tChange(0), tPress(0) {}
89  };
90  Btn btnPos(PIN_BTN_POS), btnUp(PIN_BTN_UP), btnDown(PIN_BTN_DOWN), btnRel(PIN_BTN_REL);
91
92  void readBtn(Btn &b, uint32_t now) {
93    bool raw = (digitalRead(b.pin) == LOW);
94    b.pressedEdge = b.releasedEdge = false;
95    if (raw != b.lastRaw) { b.lastRaw = raw; b.tChange = now; }
96    if ((now - b.tChange) >= DEBOUNCE_MS && raw != b.state) {
97      b.state = raw;
98      if (raw) { b.pressedEdge = true; b.tPress = now; }
99      else     { b.releasedEdge = true; }
100   }
101 }
102
103
```

CONSTANTES DE CALIBRACIÓN

- Geometría: A_CM, B_CM, SPOOL_R_CM.
- Servos: rangos (270° y 180°) y los pulsos de 500 a 2500 µs.
- Ángulos del codo: reposo 50°, mínimo 60°, máximo 120°.
- Velocidades: 15°/s en asistencia y 25°/s al liberar.
- Protecciones y tiempos: límite de corriente, tiempos de falla, ciclo de 20 ms y antirrebote de 30 ms.

ELECTRÓNICA Y SENSORES

```
102
103
104 struct Beeper { uint8_t left; uint16_t onMs, offMs; bool on; uint32_t t; } bz = {0, 0, 0, false, 0};
105
106 void beep(uint8_t n, uint16_t onMs, uint16_t offMs = 100) {
107   bz.left = n; bz.onMs = onMs; bz.offMs = offMs; bz.on = false; bz.t = 0;
108 }
109
110 void updateBeeper(uint32_t now) {
111   if (bz.on) {
112     if (now - bz.t >= bz.onMs) {
113       digitalWrite(PIN_BUZZ, LOW);
114       bz.on = false; bz.t = now;
115       if (bz.left) bz.left--;
116     }
117   } else if (bz.left) {
118     if (bz.t == 0 || now - bz.t >= bz.offMs) {
119       digitalWrite(PIN_BUZZ, HIGH);
120       bz.on = true; bz.t = now;
121     }
122   }
123 }
124
125
126 void updateLed(uint32_t now) {
127   bool on;
128   if (lockout)                on = (now / 100) % 2;
129   else switch (state) {
130     case ST_REST:    on = (now % 2000) < 60; break;
131     case ST_ASSIST:  on = atLimit ? ((now / 125) % 2) : true; break;
132     case ST_FAULT:   on = (now / 100) % 2; break;
133     default:         on = (now / 500) % 2; break;
134   }
135   digitalWrite(PIN_LED, on ? HIGH : LOW);
```

ESTADO GLOBAL

- ST_REST
- Reposo: cable y banda flojos
- ST_GO_ROT
- Rotando la banda a la posición funcional
- ST_GO_ELBOW
- Flexionando el codo hasta el ángulo funcional
- ST_ASSIST
- Asistencia: S2 y S3 mueven el codo
- ST_RELEASING
- Liberando: vuelve a reposo
- ST_FAULT
- Falla por sobrecorriente

ELECTRÓNICA Y SENSORES

ORTESIS_SERVOS_copy_20261104153844.ino

```
135   digitalWrite(PIN_LED, on ? HIGH : LOW);
136 }
137
138
139 float cableLen(float thetaDeg) {
140   float t = thetaDeg * DEG_TO_RAD;
141   return sqrtf(A_CM * A_CM + B_CM * B_CM + 2.0f * A_CM * B_CM * cosf(t));
142 }
143
144
145 float elbowServoDeg(float theta) {
146   return (cableLen(THETA_REST) - cableLen(theta)) / SPOOL_R_CM * RAD_TO_DEG;
147 }
148
149 void writeServoDeg(Servo &s, float deg, float range) {
150   deg = constrain(deg, 0.0f, range);
151   s.writeMicroseconds(SERVO_US_MIN + (int)(deg / range * (SERVO_US_MAX - SERV
152 }
153
154 void drive() {
155   if (!servosAttached) return;
156   writeServoDeg(servoE, elbowServoDeg(elbowCur), ELBOW_SERVO_RANGE_DEG);
157   writeServoDeg(servoR, rotCur, ROT_SERVO_RANGE_DEG);
158 }
159
160
```

BOTONES

- Un struct agrupa lo que cada botón necesita recordar. El constructor Btn(uint8_t p) recibe el pin y deja los demás campos en cero o falso. Con eso, btnPos, btnUp, btnDown y btnRel son cuatro botones independientes.
- readBtn hace esto, en orden:
- Lee el pin y lo invierte (== LOW), porque pulsado es LOW.
- Borra los flancos de la vuelta anterior.
- Si la lectura cambió, anota cuándo cambió (tChange).
- Si el valor lleva estable 30 ms y es distinto del estado oficial, lo acepta, actualiza state y marca el flanco que corresponda.

ELECTRÓNICA Y SENSORES

```
160
161  void attachServos() {
162    if (servosAttached) return;
163    servoE.setPeriodHertz(50);
164    servoR.setPeriodHertz(50);
165    servoE.attach(PIN_SERVO_CODO, SERVO_US_MIN, SERVO_US_MAX);
166    servoR.attach(PIN_SERVO_ROT, SERVO_US_MIN, SERVO_US_MAX);
167    elbowCur = elbowTarget = THETA_REST;
168    rotCur = rotTarget = ROT_REST_DEG;
169    servosAttached = true;
170    drive();
171  }
172
173  bool moveToward(float &cur, float target, float speed, float dt) {
174    float d = target - cur, step = speed * dt;
175    if (fabsf(d) <= step) { cur = target; return true; }
176    cur += (d > 0) ? step : -step;
177    return false;
178  }
179
180
181  void saveConfig() {
182    prefs.putFloat("thF", thetaFunc);
183    prefs.putFloat("rotF", rotFunc);
184    prefs.putFloat("iLim", iLimitMa);
185  }
186
187  void showConfig() {
188    Serial.printf("Config: codo funcional=%.1f°  rotación funcional=%.1f°  límite=%.0f mA\n",
189                  thetaFunc, rotFunc, iLimitMa);
190    Serial.printf("Servo codo: reposo 0° -> 60° de flexión = %.0f° -> 120° = %.0f° (rango %.0f°)\n",
191                  elbowServoDeg(THETA_MIN), elbowServoDeg(THETA_MAX), ELBOW_SERVO_RANGE_DEG);
192  }
193
```

EL ZUMBADOR

- beep(n, onMs, offMs) solo apunta lo que quieres: n pitidos, cuánto dura cada uno y la pausa entre ellos. No suena nada todavía.
- updateBeeper(now) corre en cada vuelta y hace sonar o callar el pin según el reloj:
- Si está sonando y ya pasó onMs, lo apaga y descuenta un pitido.
- Si está callado, quedan pitidos y ya pasó offMs, lo enciende.

ELECTRÓNICA Y SENSORES

ORTESIS_SERVOS_copy_20261104153844.ino

```
193
194  void parseCmd(char *c) {
195    if (!strcmp(c, "show"))              showConfig();
196    else if (!strncmp(c, "fe ", 3))      { thetaFunc = constrain((float)atof(c + 3), THETA_MIN, THETA_MAX); showConfig(); }
197    else if (!strncmp(c, "fr ", 3))      { rotFunc = constrain((float)atof(c + 3), ROT_REST_DEG, ROT_MAX_DEG); showConfig(); }
198    else if (!strncmp(c, "ilim ", 5))    { iLimitMa = constrain((float)atof(c + 5), 200.0f, 3000.0f); showConfig(); }
199    else if (!strcmp(c, "save"))         { saveConfig(); Serial.println("Guardado."); }
200    else Serial.println("Comandos: show | fe <60-120> | fr <0-90> | ilim <mA> | save");
201  }
202
203  void handleSerial() {
204    static char buf[32];
205    static uint8_t n = 0;
206    while (Serial.available()) {
207      char ch = (char)Serial.read();
208      if (ch == '\n' || ch == '\r') {
209        if (n) { buf[n] = 0; parseCmd(buf); n = 0; }
210      } else if (n < sizeof(buf) - 1) {
211        buf[n++] = ch;
212      }
213    }
214  }
215
216
217  void enterFault(const char *why) {
218    state = ST_FAULT;
219    elbowTarget = THETA_REST;
220    rotTarget = ROT_REST_DEG;
221    overMs = 0;
222    beep(3, 300, 150);
223    Serial.printf("FALLA: %s (%.0f mA, riel %.2f V). Retrocediendo a reposo; pulsa S4 para salir.\n",
224                  why, currentMa, railV);
225  }
226
```

EL LED

- now % 2000 < 60: de cada 2000 ms, el LED está encendido los primeros 60. Es un destello corto cada 2 s.
- (now / 500) % 2: el cociente cambia cada 500 ms y el % 2 lo vuelve 0 o 1, así que parpadea a 1 Hz.

ELECTRÓNICA Y SENSORES

ORTESIS_SERVOS_copy_20261104153844.ino

```
}

void measure(float dt) {
  float dtMs = dt * 1000.0f;
  if (inaOk) {
    currentMa = ina.getCurrent_mA();
    railV = ina.getBusVoltage_V();
  } else {
    currentMa = 0;
    railV = ALLOW_NO_INA219 ? 6.0f : 0.0f;
  }
  railOk = (railV >= RAIL_MIN_V);
  if (!servosAttached || !inaOk) return;


  if (!railOk) {
    railLostMs += dtMs;
    if (railLostMs >= RAIL_LOST_MS && !railLost) {
      railLost = true;
      state = ST_REST;
      elbowCur = elbowTarget = THETA_REST;
      rotCur = rotTarget = ROT_REST_DEG;
      beep(2, 150, 120);
      Serial.println("Riel de servos caído: reposo. Esperando que vuelva
    }
  } else {
    railLostMs = 0;
    railLost = false;
  }


  if (railOk && fabsf(currentMa) > iLimitMa) overMs += dtMs;
  else overMs = fmaxf(0.0f, overMs - 2.0f * dtMs);
  if (overMs >= OVERCURRENT_MS && state != ST_FAULT) enterFault("sobrecorriente");
}
```

GEOMETRÍA Y SERVOS

- Las cuatro funciones se encadenan:
- cableLen(θ): longitud del cable con la ley de cosenos.
- elbowServoDeg(θ): (cableLen(THETA_REST) - cableLen(θ)) / SPOOL_R_CM * RAD_TO_DEG. Es el cable que hay que recoger dividido por el radio, es decir, el giro del carrete en radianes. RAD_TO_DEG lo pasa a grados.
- writeServoDeg(servo, deg, range): convierte grados en microsegundos y se los manda al servo.
- drive(): llama a la anterior para los dos servos, solo si ya están acoplados.
- attachServos() hace el acople la primera vez y deja ambos ejes en su posición de reposo.

ELECTRÓNICA Y SENSORES

```
260
261 void requestFunctional() {
262   if (lockout || state == ST_FAULT || !railOk) { beep(2, 60, 60); return; }
263   attachServos();
264   state = ST_GO_ROT;
265 }
266
267 void requestRelease() {
268   if (!servosAttached || state == ST_REST) return;
269   state = ST_RELEASING;
270 }
271
272 void handleButtons(uint32_t now) {
273   if (lockout) return;
274
275   if (btnRel.pressedEdge) {
276     if (state == ST_FAULT) {
277       bool atRest = (elbowCur == THETA_REST && rotCur == ROT_REST_DEG);
278       if (atRest && fabsf(currentMa) < iLimitMa) { state = ST_REST; beep(1,
279     } else {
280       requestRelease();
281     }
282   }
283
284   if (btnPos.pressedEdge) posLongDone = false;
285   if (btnPos.state && !posLongDone && state == ST_ASSIST && (now - btnPos.tPress) >= LONG_MS) {
286     posLongDone = true;
287     thetaFunc = constrain(elbowCur, THETA_MIN, THETA_MAX);
288     saveConfig();
289     beep(3, 60, 60);
290     Serial.printf("Posición funcional guardada: %.1f°\n", thetaFunc);
291   }
292   if (btnPos.releasedEdge && !posLongDone && (now - btnPos.tPress) < LONG_MS) requestFunctional();
293 }
```

MEMORIA Y PUERTO SERIE

- saveConfig() guarda tres valores con prefs.putFloat. En setup() se leen con getFloat, y el segundo argumento es el valor por defecto.
- parseCmd(char *c) compara el texto recibido:
- strcmp devuelve 0 si son iguales, por eso se escribe !strcmp(...).
- strncmp(c, "fe ", 3) compara solo los 3 primeros caracteres.
- atof(c + 3) convierte lo que sigue (por ejemplo "100") a número.
- handleSerial() guarda lo que llega en buf, que es static para que no se borre entre llamadas. Cuando recibe Enter, cierra el texto y llama a parseCmd.

ELECTRÓNICA Y SENSORES

ORTESIS_SERVOS_copy_20261104153844.ino

```
293  }
294
295  void runState(float dt) {
296    float eSp = ELBOW_SPEED, rSp = ROT_SPEED;
297    switch (state) {
298      case ST_GO_ROT:   rotTarget = rotFunc; break;
299      case ST_GO_ELBOW: elbowTarget = thetaFunc; break;
300      case ST_ASSIST:
301        if (btnUp.state && !btnDown.state)      elbowTarget = THETA_MAX;
302        else if (btnDown.state && !btnUp.state) elbowTarget = THETA_MIN;
303        else                                    elbowTarget = elbowCur;
304        break;
305      case ST_RELEASING:
306      case ST_FAULT:
307        elbowTarget = THETA_REST; rotTarget = ROT_REST_DEG;
308        eSp = ELBOW_REL_SPEED; rSp = ROT_REL_SPEED;
309        break;
310      default: break;
311    }
312
313    bool eDone = moveToward(elbowCur, elbowTarget, eSp, dt);
314    bool rDone = moveToward(rotCur, rotTarget, rSp, dt);
315
316    if (state == ST_GO_ROT && rDone)                    state = ST_GO_ELBOW;
317    else if (state == ST_GO_ELBOW && eDone)             { state = ST_ASSIST; beep(1, 60); }
318    else if (state == ST_RELEASING && eDone && rDone)   state = ST_REST;
319
320
321    atLimit = (state == ST_ASSIST) &&
322              ((btnUp.state && !btnDown.state && elbowCur >= THETA_MAX - 0.01f) ||
323               (btnDown.state && !btnUp.state && elbowCur <= THETA_MIN + 0.01f));
324    if (atLimit && !limitBeeped) { beep(1, 40); limitBeeped = true; }
325    if (!btnUp.state && !btnDown.state) limitBeeped = false;
326  }
327
```

LA LÓGICA

enterFault(why): pone el estado en falla, fija el objetivo en reposo, reinicia el contador y pita tres veces.

requestFunctional() y requestRelease(): son las órdenes de S1 y S4. La primera rechaza con dos pitidos si hay bloqueo, falla o el riel está caído. Si acepta, acopla los servos y pone el estado en ST_GO_ROT.
handleButtons(now): interpreta los flancos.
runState(dt): es el cerebro y trabaja en dos tiempos. Primero, un switch fija los objetivos según el estado. Después, moveToward mueve los dos ejes, y según si llegaron (eDone, rDone) se encadenan los estados. Al final avisa con un pitido si llegaste a un tope con el botón pulsado.

ELECTRÓNICA Y SENSORES

ORTESIS_SERVOS_copy_20261104153844.ino

```
328
329  void setup() {
330    Serial.begin(115200);
331    pinMode(PIN_BTN_POS, INPUT_PULLUP);
332    pinMode(PIN_BTN_UP, INPUT_PULLUP);
333    pinMode(PIN_BTN_DOWN, INPUT_PULLUP);
334    pinMode(PIN_BTN_REL, INPUT_PULLUP);
335    pinMode(PIN_LED, OUTPUT);
336    pinMode(PIN_BUZZ, OUTPUT);
337    digitalWrite(PIN_BUZZ, LOW);
338
339    ESP32PWM::allocateTimer(0);
340    ESP32PWM::allocateTimer(1);
341
342    Wire.begin(PIN_SDA, PIN_SCL);
343    inaOk = ina.begin();
344    if (inaOk) ina.setCalibration_32V_2A();
345
346    prefs.begin("ortesis", false);
347    thetaFunc = constrain(prefs.getFloat("thF", 90.0f), THETA_MIN, THETA_MAX);
348    rotFunc   = constrain(prefs.getFloat("rotF", 40.0f), ROT_REST_DEG, ROT_MAX_DEG);
349    iLimitMa  = prefs.getFloat("iLim", I_LIMIT_DEFAULT_MA);
350
351    Serial.println("\nÓrtesis con servos - ESP32");
352    showConfig();
353    if (elbowServoDeg(THETA_MAX) > ELBOW_SERVO_RANGE_DEG) {
354      lockout = true;
355      Serial.println("ERROR: el carrete necesita más giro del que da el servo del codo. Aumenta SPOOL_R_CM o usa un servo de más rango.")
356    }
357    if (!inaOk && !ALLOW_NO_INA219) {
358      lockout = true;
359      Serial.println("ERROR: no se encuentra el INA219 (I2C 0x40). Servos deshabilitados.");
360    }
361    beep(lockout ? 3 : 1, 100, 100);
362  }
```

SETUP() Y LOOP()

setup() prepara los pines y el I2C, crea el INA219, carga lo guardado y comprueba dos cosas: que el carrete cabe en el servo y que el sensor responde. Si alguna falla, activa lockout.
loop() es una cadena de pasos:
Siempre: zumbador → LED → puerto serie Cada 20 ms: leer botones → medir → atender botones → ejecutar estado → mover servos

ELECTRÓNICA Y SENSORES

UNIVERSIDAD PERUANA CAYETANO HEREDIA

PUCP

```
362  }
363
364  void loop() {
365    uint32_t now = millis();
366    updateBeeper(now);
367    updateLed(now);
368    handleSerial();
369
370    if (now - lastTick < LOOP_MS) return;
371    float dt = (now - lastTick) / 1000.0f;
372    lastTick = now;
373
374    readBtn(btnPos, now); readBtn(btnUp, now); readBtn(btnDown, now); readBtn(btnRel, now);
375    measure(dt);
376    handleButtons(now);
377    runState(dt);
378    drive();
379
380    if (now - lastStatus >= 1000) {
381      lastStatus = now;
382      Serial.printf("[%s] codo=%.1f° rot=%.0f° I=%.0f mA riel=%.2f V\n",
383                    STATE_NAME[state], elbowCur, rotCur, currentMa, railV);
384    }
385  }
386
```

VERIFICACIÓN PRELIMINAR DEL DISEÑO POR SUBSISTEMAS

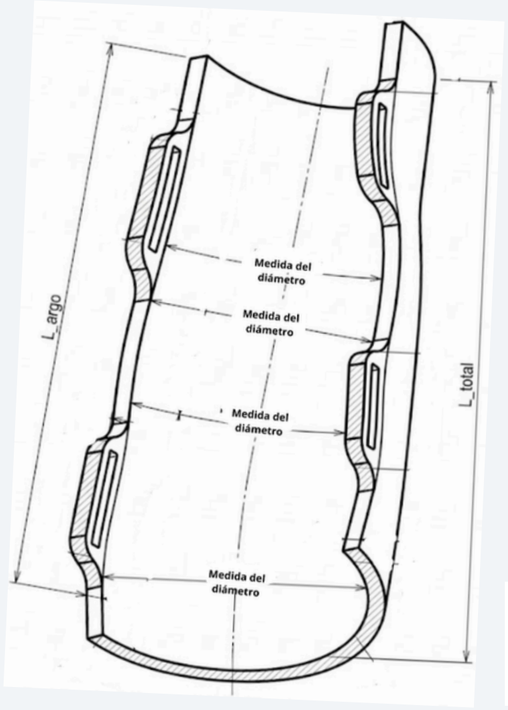
MECÁNICA Y FABRICACIÓN

Tipos de verificación de dimensiones:

1. **Verificación de los tamaños de piezas** 3D
2. **Ajuste las piezas** impresas
3. Prueba de elementos: **organizar parte electrónica**
4. Prueba de movimiento.

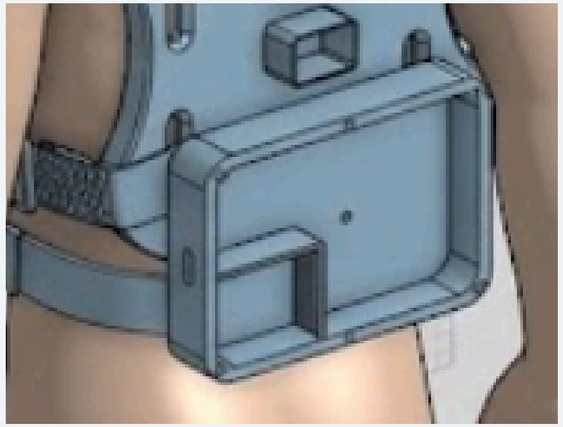
Las evidencias:

- Foto de **ajustes de pieza** y del cable en la ranura.
- Registro del **peso aproximado**
- **Recorrido real** frente al calculado/esperado (considerando ángulos)

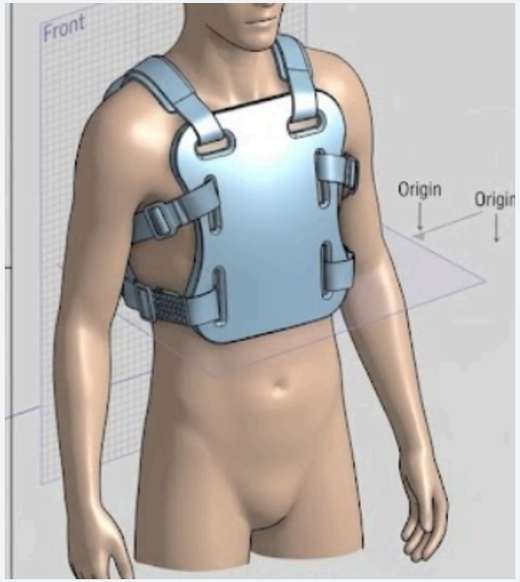


Medida	Diámetro (cm)	Contorno (cm)
1	6,5	23
2	6,5	23
3	6,5	21,5
4	5,5	20,5
5	5,5	19,5
6	5,7	17,5
7	5,5	16.8
8	5,5	15,7

Medida	Largo del antebrazo (cm)
1	23cm



Organización de parte electronica dentro de la caja



MECÁNICA Y FABRICACIÓN

Requisitos de Tolerancia

± 0.15 mm

Tolerancia FDM Fina

Ajuste Dimensional 3D

Rango de precisión aceptado en ejes y alojamientos para evitar juego mecánico u holguras no deseadas.

220 °C

Inserción Térmica

Insertos Roscados (M2.5 / M3)

Ajuste de interferencia en tuercas de latón para resistir torques dinámicos sin desprendimiento.

Holgura

Paso de Cinchas

Ajuste Ergonómico Textil

Tolerancia abierta en ranuras para asegurar deslizamiento continuo de las correas de velcro.

MECÁNICA Y FABRICACIÓN

Topes Mecánicos

LÍMITE ABDUCCIÓN

Máx 80°

(Restricción clínica inicial: 40°)

LÍMITE ROTACIÓN EXTERNA

Máx 45°

(Restricción clínica inicial: 30°)

Protección Biomecánica Pasiva

- **Función:** Evita subluxaciones glenohumerales o distensiones ante fallos de control.
- **Estructura Física:** Bloqueadores rígidos de PETG/PLA+ integrados directamente en las articulaciones.
- **Validación:** Pruebas de carga estática con goniómetro directo bajo el torque nominal de los servomotores.

ELECTRÓNICA Y SENSORES

UNIVERSIDAD PERUANA CAYETANO HEREDIA

PUCP

Verificación de voltaje

Comprobar la alimentación eléctrica y las conexiones de corriente estando estables y dentro de los rangos seguros para los componentes del dispositivo

Verificación de sensores

Validar la correcta lectura de las señales analógicas y digitales del microcontrolador

Verificación de Actuadores

Confirmar las respuestas de manera fisica y precisa de los servomotores y mecanismos del dispositivo

Verificación de conexión y pantallas

Comprobar la vinculación vía Bluetooth y la respuesta del aplicativo

Verificación de entradas

Verificar la correcta lectura de las señales de los botones - “Arriba”, “Abajo”, “Soltar” y “Posición funcional”

Control de actuadores

Comprobar la ejecución de la rotación y el movimiento asistido del codo mediante los servomotores según el comando enviado

Prueba de lógica de seguridad y sobrecarga

Simular una lectura del sensor de tensión superior al límite establecido para comprobar que el sistema detiene automáticamente el movimiento

PROGRAMACIÓN Y CONTROL

UNIVERSIDAD PERUANA CAYETANO HEREDIA

PUCP

1. FIRMWARE ESP32

Lógica de Control

Controlador central (ESP32 DevKit) gestiona lazo cerrado, IMU y actuadores.

- Sincronización Bluetooth con App.
- Lectura inicial IMU (MPU-6050).
- Protección activa ante sobrecorriente.

2. ENTRADAS (INPUTS)

Señales y Comandos

Captura de estados de conexión y botonera interactiva:

- Estado Bluetooth (Sí/No).
- Botones: Conectar, Posición Funcional, Flexionar, Soltar.
- Sensores: Seguridad y límites (OK/Error).

3. SALIDAS (OUTPUTS)

Acciones y App

Respuesta ejecutada en actuadores y pantallas:

- Pantallas 1 y 2 (Registro y ángulos).
- Señales PWM a motores (35 y 20 kg·cm).
- Parada, liberación de cable y alarmas.